



SME0211 - Otimização Linear (2024)



Nome: Número USP
Francisco Maian: 14570890
Gustavo Moreira: 5244057
Julia Pravato: 14615054
Murilo Lirani: 11234673

Contents

Contents	2
1 Introdução	3
1.1 Conceitos Básicos	3
1.2 Balanceamento de Carga Problema de Otimização em Matriz Ponderada	4
1.3 Descrição do Problema	4
1.4 Formulação do Problema de Otimização Linear	4
1.4.1 Variáveis de Decisão	4
1.4.2 Função Objetivo	4
1.4.3 Restrições	5
1.5 Reescrita do Problema	5
1.6 Simplex Revisado	6
1.7 Simplex com a Regra de Bland	7
1.8 Análise de Tamanho do Problema	8
2 Implementação computacional	9
2.1 Função <code>calc_beta_params</code>	9
2.2 Função <code>gerar_tensor_aleatorio</code>	9
2.3 Função <code>traducao_variaveis</code>	9
2.4 Função <code>forma_padrao</code>	10
2.5 Função <code>simplex</code>	10
2.6 Uso do <code>linprog</code>	11
3 Experimentos	11
3.1 Função <code>teste_singular</code> :	11
3.2 Função <code>amostragem</code> :	12
3.3 Função <code>experimentos</code> :	13
4 Resultados	13
4.1 Considerações sobre a Implementação em Python	14
4.2 Tempos de Execução e Número de Iterações Médios por algoritmo	14
4.3 Correlação	15
4.4 Variante: Size	16
4.5 Variante: Sparsity	17
4.6 Teste de Escalabilidade do Problema	18
5 Conclusão	18
References	19

1 Introdução

A otimização linear é uma ferramenta fundamental na tomada de decisões em diversas áreas como logística, produção, finanças e planejamento estratégico. Neste relatório, será apresentado um estudo detalhado de um problema de otimização linear, desde sua formulação matemática até a obtenção da solução ótima. Serão abordados os métodos utilizados para resolução e as análises dos resultados obtidos, para assim, exemplificar e exaltar a importância dessa ferramenta no contexto prático. Contudo, antes de apresentarmos o problema que encontraremos a solução, será feita uma introdução básica a conceitos de otimização linear.

1.1 Conceitos Básicos

A Otimização Linear é um método matemático para resolver problemas nos quais se busca maximizar ou minimizar uma função linear (chamada de *função objetivo*) sujeita a um conjunto de restrições também lineares. A função objetivo é a expressão matemática que representa o valor que se deseja otimizar (maximizar ou minimizar). Ela é geralmente escrita como uma combinação linear das variáveis, como será mostrado a seguir:

$$z = c_1x_1 + c_2x_2 + \cdots + c_nx_n = c^T x$$

Além da função objetivo, há também as **restrições** do problema, que são limites ou condições que as variáveis devem obedecer. Elas são representadas por inequações ou equações lineares, já que estamos tratando de um problema de otimização linear, assim como pode ser visto a seguir:

$$a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n \leq b_1 \quad (\text{Exemplo de possível restrição em formato de inequação})$$

Onde:

- a_{ij} : coeficiente que indica a relação entre a variável x_j e a restrição i ,
- b_i : valor limitante permitido pela restrição.

Geralmente, as restrições são representadas pela notação Ax , onde A é uma matriz com os coeficientes a_{ij} e x é o vetor das variáveis.

Com isso, é possível formular um problema de otimização linear com os elementos descritos acima. Contudo, para padronizar e facilitar a resolução desses problemas, foi definida uma **forma padrão**, que geralmente adequa o problema por meio de manipulações matemáticas e pela adição de variáveis de folga. Dessa forma, chegamos na seguinte forma padrão:

$$\begin{aligned} P : \min \quad & c^T x \\ \text{s. a} \quad & Ax = b, \\ & x \geq 0 \end{aligned}$$

Basicamente, para passar para a forma padrão, é necessário que todas as variáveis sejam não negativas, que o problema seja transformado em uma minimização (caso necessário) e que as restrições se tornem equações (caso sejam inequações).

Agora, com essas informações, será iniciada a inspiração do problema será desenvolvido nesse relatório. Como era desejado a resolução de um problema prático e de larga escala, foi especificado um problema de otimização linear para balanceamento de carga.

1.2 Balanceamento de Carga

Problema de Otimização em Matriz Ponderada

O problema descrito é um exemplo de balanceamento de carga, onde o objetivo é distribuir adequadamente tarefas a serem realizadas limitando-se aos recursos disponíveis. Esse tipo de problema pode ser modelado como um problema de otimização linear.

$$R = \begin{bmatrix} r_{11} & r_{12} & \cdots & r_{1m} \\ r_{21} & r_{22} & \cdots & r_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ r_{n1} & r_{n2} & \cdots & r_{nm} \end{bmatrix} \quad \begin{bmatrix} q_1 \\ q_2 \\ \vdots \\ q_n \end{bmatrix} = q$$
$$t = [t_1 \quad t_2 \quad \cdots \quad t_m]$$

Em geral, as quantidades de cada tarefa i a ser realizada são descritas por q_i , os tetos limites de recursos disponíveis para cada recurso j são descritas em t_j . Enquanto o valores de relevância r_{ij} pontuam o sistema por a executar uma certa tarefa i utilizando um determinado recurso j .

O objetivo do problema é aumentar a soma da relevância global respeitando os limites de recurso disponíveis. De tal maneira, o sistema tende a escolher a maior quantidade de tarefas, priorizando as mais relevantes, sem sobrecarregar o sistema.

1.3 Descrição do Problema

Considerando o problema geral citado acima, será apresentado um problema análogo, utilizando as seguintes interpretações:

- **Linhas representam as tarefas:** Cada linha na matriz representa uma tarefa ou carga de trabalho que precisa ser processada. O número de linhas corresponde ao número de tarefas que devem ser distribuídas entre os servidores.
- **Colunas representam os servidores:** Cada coluna na matriz representa um servidor ou recurso computacional que pode processar as tarefas. O número de colunas corresponde ao número de servidores disponíveis para processar as tarefas.
- **Pesos representam a relevância de processamento:** O valor na posição r_{ij} na matriz indica relevância associada a tarefa i processada pelo servidor j . Considerando apenas os valores dentro de uma linha específica qualquer q , os valores mais altos de r_{qj} representam os melhores servidores para executar tal tarefa. Considerando apenas os valores dentro de uma coluna específica qualquer t , os valores mais altos de r_{it} representam quais tarefas tem maior prioridade.

1.4 Formulação do Problema de Otimização Linear

O objetivo é maximizar a **relevância total** de alocar as tarefas.

1.4.1 Variáveis de Decisão

Definimos x_{ij} como a **porção da tarefa i** que será alocada ao **servidor j** . Essas variáveis são **não negativas** por natureza, ou seja, $x_{ij} \geq 0$.

1.4.2 Função Objetivo

A função objetivo é maximizar a **relevância total** de alocação de tarefas aos servidores. A função objetivo pode ser formulada como:

$$\min \sum_{i=1}^n \sum_{j=1}^m r_{ij} \cdot x_{ij}$$

Onde:

- n é o número de tarefas distintas.
- m é o número de servidores.
- r_{ij} é a relevância computacional de alocar a tarefa i ao servidor j .
- x_{ij} é a quantidade de tarefa i atribuída ao servidor j .

1.4.3 Restrições

- **Restrição de carga de trabalho das tarefas:** Como o problema considerado está naturalmente normalizado, uma vez que é avaliada qual a porção da quantidade de cada tarefa a ser executada em cada servidor, a soma de todas as porções da tarefa i alocadas aos servidores deve ser igual a 1 (ou inferior caso não haja mais recursos disponíveis):

$$\sum_{j=1}^m x_{ij} \leq 1, \quad \forall i = 1, 2, \dots, n$$

Foram testados casos nos quais os elementos q_i (restrições de somas para linhas) eram distintas. Entretanto, como será também na parte de resultados, não há diferença significativa no tempo de execução para tais diferenciações. Além disso, o problema é equivalente (simétrico) em relação as restrições de linhas ou colunas. Por esses motivos junto à normalização natural do problema, foi mantido $q_i = 1$ para todas as linhas.

- **Restrição de capacidade dos servidores:** O número total de tarefas alocadas a cada servidor não pode ultrapassar seu teto de processamento. Se t_j for a capacidade do servidor j , temos a restrição:

$$\sum_{i=1}^m x_{ij} \leq t_j, \quad \forall j = 1, 2, \dots, n$$

- **Restrições de não-negatividade:** As variáveis x_{ij} devem ser **não negativas**:

$$x_{ij} \geq 0, \quad \forall i = 1, 2, \dots, n \quad \forall j = 1, 2, \dots, m$$

1.5 Reescrita do Problema

Como visto, cada elemento da matriz de relevância está associado a uma variável do problema. Portanto, para escrevermos os valores x_{ij} em um único vetor, iremos considerar uma transformação (*ravel*) que concatena as linhas da matriz associada, gerando um vetor coluna de dimensão $(nm \times 1)$.

De maneira equivalente, cada relevância sofrerá a mesma transformação.

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1m} \\ x_{21} & x_{22} & \cdots & x_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nm} \end{bmatrix}$$

O ravel de X , ou seja, a concatenação dos elementos por linhas, é dado por:

$$\vec{x}^T = \text{ravel}(X)^T = [x_{11}, x_{12}, \dots, x_{1m}, x_{21}, x_{22}, \dots, x_{2m}, \dots, x_{n1}, x_{n2}, \dots, x_{nm}]$$

Com isso, é necessário reinterpretar as restrições anteriores, agora no novo formato. Cada restrição anterior, tanto verticais, quanto horizontais, serão representadas uma a uma em uma matriz $\underline{A}$, na qual cada coluna está associada a um elemento de $\text{ravel}(X)$.

Assim, cada soma horizontal ($\sum_{j=1}^m x_{ij}$) é representada por sequências de 1's de tamanho n. A seguir está a representação esquemática dessa matriz binária, logo, os valores não apresentados são todos 0.

$$\underline{A}_h = \begin{bmatrix} \text{---} 1 \text{---} & & & & \\ & \text{---} 1 \text{---} & & & \\ & & \dots & & \\ & & & \text{---} 1 \text{---} & \end{bmatrix}$$

E cada soma $\sum_{i=1}^n x_{ij}$ vertical é representada por 1's distribuídos a cada n colunas, iniciando na coluna de índice m. A seguir está a representação esquemática dessa matriz binária, logo, os valores não apresentados são todos 0.

$$\underline{A}_v = \begin{bmatrix} 1 & & & 1 & & & 1 & & \\ & 1 & & & 1 & & & 1 & \\ & & \dots & & & \dots & & & 1 \\ & & & 1 & & & 1 & & \\ & & & & 1 & & & & 1 \end{bmatrix}$$

Assim, $\underline{A}_h \vec{x} \leq \vec{1}$ e $\underline{A}_v \vec{x} \leq \vec{t}$, o que pode ser escrito como um sistema único $\underline{A} \vec{x} \leq \vec{b}$, onde $\vec{b}$ empilha $\vec{1}$ e $\vec{t}$, enquanto $\underline{A}$ empilha $\underline{A}_h$ e $\underline{A}_v$.

Assim, o problema de otimização pode ser escrito como:

$$\begin{aligned} \underline{P}: \quad & \max \quad \underline{r}^T x \\ & \text{sujeito a} \quad \underline{A}x \leq b, \\ & \quad \quad \quad x \geq 0. \end{aligned}$$

Esse problema ainda deve ser alterado para a forma padrão, adicionando variáveis de folga e alterando o objetivo para seu oposto (minimização). Obtendo, ao fim, o problema:

$$\begin{aligned} P: \quad & \min \quad c^T x \\ & \text{sujeito a} \quad Ax = b \\ & \quad \quad \quad x \geq 0 \end{aligned}$$

A solução ótima será um vetor x^* que indica como as tarefas devem ser distribuídas entre os servidores, de modo a maximizar a relevância total.

1.6 Simplex Revisado

O Simplex Revisado é uma versão mais eficiente do algoritmo Simplex clássico para resolver problemas de programação linear. O Simplex Revisado é uma abordagem iterativa que realiza os seguintes passos:

- **Inicialização:** Começa com uma solução básica viável, onde um conjunto de variáveis são escolhidas como variáveis básicas, e o restante são variáveis não básicas.
- **Representação Compacta:** Ao invés de manter uma tabela completa com todos os coeficientes de A , o Simplex Revisado utiliza a inversa de uma submatriz B da matriz das restrições, essa que é associada às variáveis básicas, já que as colunas das variáveis não básicas terão seus valores zerados durante a multiplicação de Ax .
- **Custos Reduzidos:** O algoritmo calcula os custos reduzidos das variáveis não básicas. O valor do custo reduzido em direção a uma variável, indica como a função objetivo se comportará caso seja adicionada a variável j à base básica. De forma que, custo reduzido maior que zero induz que o custo aumentará, menor que zero que diminuirá e igual a zero que manterá o mesmo custo. O custo reduzido de uma variável x_j é dado por:

$$\text{custo reduzido}(x_j) = c_j - c_B^T B^{-1} A_j$$

Onde:

- c_j é o custo associado à variável x_j ,
- c_B é o vetor de custos das variáveis básicas,
- B é a submatriz das variáveis básicas,
- A_j é a coluna da matriz das restrições associada à variável não básica x_j .
- **Seleção da Variável de Entrada:** O algoritmo escolhe a variável com o maior custo reduzido negativo. Se todos os custos forem positivos ou zero, a solução ótima é dada como encontrada.
- **Determinação da Variável de Saída:** A variável básica que sairá da solução é determinada com base na razão entre as variáveis de decisão e as mudanças na função objetivo.
- **Atualização das Matrizes:** O Simplex Revisado atualiza as matrizes associadas ao sistema de equações de forma eficiente, usando a inversa da submatriz B^{-1} de maneira incremental, para assim, reduzir a complexidade computacional.
- **Complexidade:** Não conseguiu-se achar que operações exatas a biblioteca utilizada fez, por isso, considerando que os processos são otimizados, sendo m o número de restrições e n o número de variáveis, determina-se que a complexidade varia de tal forma:
 - **Melhor Caso:** $O(m^2)$
 - **Pior Caso:** $O(mn)$

1.7 Simplex com a Regra de Bland

O Simplex com a Regra de Bland é uma modificação do algoritmo Simplex clássico que adiciona uma estratégia para evitar o fenômeno da ciclagem, um problema no qual o algoritmo pode retornar à mesma solução básica viável várias vezes, sem avançar para a solução ótima.

- **Diferenças em Relação ao Simplex Revisado**
Enquanto o Simplex Revisado trabalha de maneira eficiente com uma matriz compacta e apenas com variáveis básicas, o Simplex com a Regra de Bland foca especificamente em garantir que o algoritmo não entre em um ciclo infinito, aplicando uma regra de escolha das variáveis de entrada e saída.
- **Ciclagem no Algoritmo Simplex:** A ciclagem ocorre quando o algoritmo retorna à mesma solução básica viável em várias iterações sem progredir para a solução ótima. Embora isso seja raro, pode acontecer em problemas específicos de otimização. Para evitar isso, o Simplex com a Regra de Bland garante que o algoritmo sempre escolhe uma variável de entrada e uma variável de saída de forma determinística e sem ambiguidade.

- Regra de Bland: A Regra de Bland é uma estratégia para escolher qual variável entrar e qual sair da base, de forma a evitar ciclos:
 - Quando houver múltiplas variáveis com o mesmo custo reduzido mínimo (para maximização), a Regra de Bland seleciona a variável com o menor índice.
 - Da mesma forma, ao determinar qual variável básica sairá, a Regra de Bland escolhe a variável com o menor índice entre as que atingem o valor de zero primeiro.
 - Ao sempre escolher a variável com o menor índice, o Simplex com a Regra de Bland evita que o algoritmo oscile entre soluções básicas viáveis que são repetidas sem progresso, garantindo que a solução ótima seja eventualmente alcançada.
 - **Complexidade:** Em cada iteração, o método do simplex com a regra de bland executa operações na grandeza de $O(nm)$, sendo m o número de restrições e n o número de variáveis, pois atualiza a matriz B^{-1} de mesmo tamanho para mudar a variável de custo reduzido negativo encontrada para a base. Sendo assim, a sua complexidade não varia entre melhor e pior caso e, então, o simplex revisado sempre será melhor, ou igual ao simplex com a regra de bland no seu pior caso, em relação a complexidade.

1.8 Análise de Tamanho do Problema

O tamanho do problema será apresentado analisando o tamanho da matriz A.

Foi mostrado que cada elemento da matriz $R_{n,m}$ está associado a uma variável, logo, o número de variáveis inicial do problema é nm, portanto, o número de colunas de $\underline{A}$ é nm. Ao mesmo tempo, o número de linhas é dado pela quantidade de restrições, o que pode ser descrito pela soma das n restrições horizontais e m restrições verticais iniciais do problema. Portanto, o número de linhas de $\underline{A}$ é n+m.

$$\underline{A}_{n+m,nm}$$

Para transformação para a matriz A, na forma padrão, é necessário adicionar as variáveis de folga. Será adicionada uma para cada linha da matriz $\underline{A}$. Assim, chega-se em uma matriz na forma padrão que será usada no simplex:

$$A_{n+m,nm+n+m}$$

De maneira equivalente, caso seja considerada uma ampliação do problema atual, na qual considera-se um tensor $R_{n,m,p}$ com altura n, largura m e profundidade p, e 3 vetores de restrição associados de tamanho apropriado, chegaremos nas dimensões para a matriz padrão A, associada ao problema:

$$A_{n+m+p,nmp+n+m+p}$$

Caso consideremos matrizes R esparsas, podemos desconsiderar os elementos associados a $r_{i,j} = 0$, uma vez que tais variáveis não alteram as soluções, pois não gastariamos recursos para processar tarefas sem relevância.

Como já visto, cada $r_{i,j}$ está associado a uma variável, ou seja, uma coluna de $\underline{A}$. Assim, se consideramos que a esparsidade de R é s, com $s \in [0, 1]$. Então, descarta-se uma proporção s das colunas de $\underline{A}$. Assim, as dimensões esperadas ficam:

$$\underline{A}_{n+m,nm(1-s)}$$

$$A_{n+m,nm(1-s)+n+m}$$

Analogamente, para o caso do tensor, temos:

$$A_{n+m+p,nmp(1-s)+n+m+p}$$

Portanto, a complexidade esperada para o problema, utilizando como base as dimensões da matriz $R_{n,m}$, é notavelmente simétrica em relação a (n,m), como esperado, e dita por:

- Melhor Caso: $O((n+m)^2)$
- Pior Caso: $O((n+m)(nm(1-s) + n+m))$

2 Implementação computacional

Primeiramente, explicaremos como geramos o problema. O código gera e manipula um tensor aleatório com base em parâmetros de distribuição Beta e realiza uma transformação para um formato adequado para resolução de problemas de otimização, utilizando matrizes e vetores. Aqui está uma explicação resumida de cada parte:

2.1 Função `calc_beta_params`

: Calcula os parâmetros de uma distribuição Beta, uma distribuição de probabilidade contínuas definidas no intervalo $[0,1]$ parametrizado por dois parâmetros positivos, estes que são α e β , com base na média e no desvio padrão dados.

2.2 Função `gerar_tensor_aleatorio`

:

- **Objetivo:** Gera um tensor de relevância e limites aleatórios.
- **Parâmetros:**
 - `size`: Tamanho do tensor gerado.
 - `B_params`: Parâmetros da distribuição Beta usados para gerar os limites.
 - `sparsity`: A porcentagem de elementos a serem definidos como zero (esparsidade).
- **Processo:**
 - Usa a distribuição Beta para gerar os limites de cada dimensão (com o parâmetro `B_params`).
 - Gera um tensor de relevâncias aleatórias entre $(0, 1)$ e aplica a esparsidade, definindo uma fração de elementos como zero.
 - Retorna o tensor de relevância e os limites.

2.3 Função `traducao_variaveis`

:

- **Objetivo:** Transforma o tensor de relevância e limites em uma forma mais utilizável para otimização.
- **Parâmetros:**
 - `relevancia`: Tensor de relevância gerado pela função `gerar_tensor_aleatorio`.
 - `limites`: Limites gerados pela função `gerar_tensor_aleatorio`.
- **Processo:**
 - A função "raveliza" ("achata") o tensor `relevancia` em um vetor.
 - Para cada dimensão do tensor, cria uma matriz S que mapeia cada elemento da dimensão para uma representação linear (vetorial).
 - Empilha todas essas matrizes S verticalmente para formar a matriz A , que é a representação linear das relações entre variáveis.
 - A função também empilha os limites em um vetor b .
 - Retorna a matriz A , o vetor b , e o vetor r (a versão "descompactada" de `relevancia`).

$$\begin{aligned} \underline{P}: \quad & \max \quad \underline{r}^T x \\ & \text{sujeito a } \underline{A}x \leq b, \\ & \quad \quad x \geq 0. \end{aligned}$$

Após isso, o problema será transformado para a forma padrão na seguinte função:

2.4 Função forma_padrao

:

- **Objetivo:** Transforma o problema de otimização linear em sua forma padrão, ajustando as variáveis e as restrições para um formato adequado para a resolução de problemas de otimização linear.
- **Parâmetros:**
 - **A_:** Matriz de coeficientes do problema de otimização linear.
 - **b_:** Vetor de limites das restrições do problema.
 - **r_:** Vetor de custos da função objetivo.
- **Processo:**
 - A função começa retirando as variáveis de custo nulo, utilizando uma máscara para identificar quais elementos de **r_** são diferentes de zero.
 - Em seguida, a matriz **A** é ajustada, concatenando uma matriz identidade I_m à matriz original de coeficientes **A_**, representando as variáveis de folga.
 - O vetor de custo **c** é ajustado, negando os valores de **r_** (para refletir uma minimização) e adicionando zeros correspondentes às variáveis de folga.
 - O vetor de limites **b** é mantido inalterado.
 - Retorna a matriz **A**, o vetor **b**, e o vetor **c**.

Após passar o problema para forma padrão, duas abordagens foram seguidas para resolução desse problema, o SimpleX Revisado usando a biblioteca SciPy e SimpleX com a Regra de Bland para evitar ciclagem.

2.5 Função simplex

:

- **Objetivo:** Implementar o algoritmo Simplex para resolver problemas de otimização linear, utilizando a regra de Bland para evitar a ciclagem.
- **Parâmetros:**
 - **c:** Vetor de coeficientes da função objetivo, representando os custos das variáveis.
 - **A:** Matriz de coeficientes das restrições do problema.
 - **b:** Vetor de limites das restrições do problema.
- **Processo:**
 - O algoritmo começa com a construção do tableau, que é uma tabela que inclui os coeficientes das variáveis, as variáveis de folga e as variáveis base.
 - Inicializa as variáveis de base com as variáveis de folga, que são aquelas que não aparecem na função objetivo.

- Enquanto houver custos negativos, o algoritmo continua. O objetivo é identificar o índice da variável que entrará na base (variável com custo negativo) e a variável que sairá da base (variável base com o menor custo positivo).
- A **Regra de Bland** é aplicada para evitar ciclos, garantindo que sempre se escolha a variável de menor índice quando houver empate na troca de variáveis.
- O tableau é atualizado a cada iteração, com as variáveis da base sendo trocadas de acordo com o critério da Regra de Bland.
- O algoritmo continua até que todos os custos reduzidos sejam não-negativos, indicando que a solução ótima foi alcançada.
- Retorna o valor da função objetivo ótima, a solução das variáveis e o número de iterações realizadas.

2.6 Uso do linprog

:

- O método `linprog` é utilizado para resolver o problema de otimização linear de forma automática usando a biblioteca `scipy`.
- A função `linprog` recebe como parâmetros:
 - `c`: Vetor de coeficientes da função objetivo.
 - `A_eq`: Matriz de coeficientes das restrições de igualdade.
 - `b_eq`: Vetor de limites das restrições de igualdade.
 - `bounds`: O intervalo em que as variáveis podem variar, nesse caso $(0, \infty)$, já que as variáveis não podem assumir valores negativos.
 - `method`: O método utilizado para resolver o problema, no caso, o método simplex.
- A função `linprog` retorna a solução ótima para o problema de otimização.

3 Experimentos

Para avaliar e comparar os dados obtidos e as solução encontradas, foram feitos testes para a avaliação dos resultados. Foi implementado um sistema para gerar problemas de otimização linear a partir de tensores esparsos e aleatórios, e compara a performance de dois métodos de resolução e também avaliar os impactos de diferentes parâmetros (dimensão, distribuição Beta e esparsidade) no desempenho.

3.1 Função teste_singular:

- **Objetivo:** Testar a solução de um problema de otimização linear gerado aleatoriamente, comparando o desempenho do método `linprog` do `scipy` com uma implementação personalizada do algoritmo Simplex.
- **Parâmetros:**
 - `size`: Dimensão do tensor a ser gerado.
 - `B_params`: Parâmetros da distribuição Beta usada para gerar os limites.
 - `sparsity`: Taxa de esparsidade, ou seja, a fração de elementos do tensor de relevância que será zerada.
- **Processo:**

- Gera o tensor de relevância e limites usando a função `gerar_tensor_aleatorio`.
- Converte o tensor gerado para o formato matricial utilizando a função `traducao_variaveis`.
- Transforma o problema para sua forma padrão com a função `forma_padrao`, gerando a matriz A , vetor de limites b e o vetor de custo c .
- Calcula o número de variáveis, `varsize`, como a diferença entre o número de colunas e o número de linhas em A .
- Resolve o problema de otimização linear utilizando:
 - * `linprog` do `scipy`, medindo o custo final (`custo_scipy`), número de iterações (`nit_scipy`), e o tempo de execução (`tempo_scipy`).
 - * Algoritmo Simplex personalizado, retornando o custo final (`custo_SimpleXBland`), número de iterações (`nit_SimpleXBland`), e o tempo de execução (`tempo_SimpleXBland`).

- **Resultados Retornados:**

- `size`: Dimensão do tensor.
- `B_params`: Parâmetros da distribuição Beta.
- `custo_scipy`: Valor da função objetivo obtido pelo `linprog`.
- `nit_scipy`: Número de iterações do `linprog`.
- `tempo_scipy`: Tempo de execução do `linprog`.
- `custo_SimpleXBland`: Valor da função objetivo obtido pelo Simplex personalizado.
- `nit_SimpleXBland`: Número de iterações do Simplex personalizado.
- `tempo_SimpleXBland`: Tempo de execução do Simplex personalizado.
- `sparsity`: Taxa de esparsidade do tensor.
- `varsize`: Número de variáveis consideradas.

3.2 Função amostragem:

- **Objetivo:** Realizar múltiplas execuções da função `teste_singular`, agregando os resultados em um `DataFrame` para análise.

- **Parâmetros:**

- `sample_size`: Número de execuções (amostras) da função `teste_singular`.
- `size`: Dimensão do tensor gerado em cada teste (valor padrão: (6, 10)).
- `B_params`: Parâmetros da distribuição Beta usada para gerar os limites (valor padrão: (0.5, 0.1)).
- `sparsity`: Taxa de esparsidade aplicada ao tensor de relevância (valor padrão: 0).

- **Processo:**

- Cria uma lista vazia `results` para armazenar os resultados de cada execução da função `teste_singular`.
- Realiza um loop com `sample_size` iterações, chamando `teste_singular` e adicionando os resultados à lista `results`.
- Define as colunas do `DataFrame` para armazenar os dados retornados pela função `teste_singular`:
 - * `size`: Dimensão do tensor.
 - * `B_params`: Parâmetros Beta.
 - * `custo_scipy`, `nit_scipy`, `tempo_scipy`: Resultados obtidos pelo `linprog`.
 - * `custo_SimpleXBland`, `nit_SimpleXBland`, `tempo_SimpleXBland`: Resultados obtidos pelo Simplex manual.
 - * `sparsity`: Taxa de esparsidade do tensor.

- * **varsize**: Número de variáveis consideradas.
 - Converte **results** para um **DataFrame** utilizando as colunas definidas.
 - Expande os valores do campo **B_params** em duas colunas separadas (**B0** e **B1**).
 - Calcula a dimensão do tensor gerado (quantidade de dimensões) e armazena na coluna **tensor_dim**.
 - Remove a coluna **B_params**, pois seus valores já foram separados em **B0** e **B1**.
 - Retorna o **DataFrame** final para análise.
- **Resultado Retornado**: Um **DataFrame** com as seguintes colunas:
 - **size**: Dimensão do tensor.
 - **custo_scipy**, **nit_scipy**, **tempo_scipy**: Resultados obtidos pelo **linprog**.
 - **custo_SimplexBland**, **nit_SimplexBland**, **tempo_SimplexBland**: Resultados obtidos pelo Simplex personalizado.
 - **sparsity**: Taxa de esparsidade do tensor.
 - **varsize**: Número de variáveis consideradas.
 - **B0**, **B1**: Parâmetros Beta usados.
 - **tensor_dim**: Quantidade de dimensões do tensor gerado.

3.3 Função experimentos:

- **Objetivo**: Realizar múltiplas execuções da função **amostragem** variando os parâmetros de entrada (**size**, **B_params**, **sparsity**) e consolidar os resultados em um único **DataFrame**.
- **Parâmetros**:
 - **sample_size**: Número de amostras a serem geradas em cada execução.
 - **sizeS**: Lista de dimensões de tensores para testar
 - **beta_paramsS**: Lista de pares de parâmetros para a distribuição Beta
 - **sparsityS**: Lista de taxas de esparsidade para testar diferentes configurações de densidade nos tensores.
- **Processo**:
 - Para cada valor em **sizeS**, executa a função **amostragem**, armazenando os resultados e o tempo de execução.
 - Para cada par de parâmetros Beta em **beta_paramsS**, executa a função **amostragem**, armazenando os resultados e o tempo de execução.
 - Para cada taxa de esparsidade em **sparsityS**, executa a função **amostragem**, armazenando os resultados e o tempo de execução.
 - Consolida todos os **DataFrames** gerados em um único **DataFrame**.
- **Retorno**: Um **DataFrame** contendo os resultados de todas as execuções, com as colunas ajustadas para refletir os parâmetros testados e os resultados obtidos.

4 Resultados

Em questão da exploração e análise do desempenho sobre esse experimento aplicando o algoritmo implementado a fim de concluirmos resultados numa visão de otimização linear, vamos não só coletar os dados do experimento para características intrínsecas do problema, mas principalmente comparar com o desempenho do modelo disponibilizado na biblioteca Scipy. Dessa forma, teremos base para parametrizar eventos nos resultados do modelo criado.

O principal ponto demonstrado deve ser em torno da eficiência computacional dos algoritmos e escalabilidade, portanto, não depender apenas do tamanho absoluto do problema, mas também em outros parâmetros que possam afetar os tempos: distribuição dos coeficientes nas restrições por coluna, a esparsidade das matrizes, formato da matriz, entre outros. Essa análise desempenha um papel central para compreender a eficiência, escalabilidade e robustez (estrutura) das abordagens utilizadas no design de métodos otimizados para diferentes contextos.

Para as amostragens, os parâmetros do problema foram alterados, gerando diferentes amostras aleatórias que tiveram seus resultados comparados entre si. Foi mantida uma *seed* fixa para as amostragens geradas, possibilitando a replicabilidade dos resultados, apesar dos tempos diferirem a cada execução. Todos os experimentos foram executados no ambiente do Google Colaboratory.

Nesta seção, apresentamos uma análise detalhada dos resultados obtidos a partir dos experimentos.

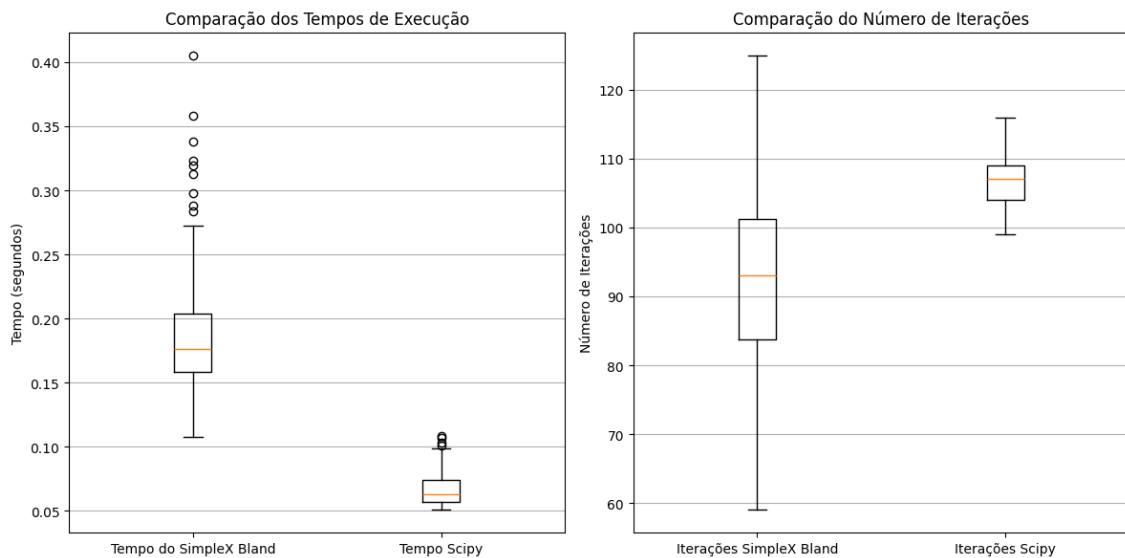
4.1 Considerações sobre a Implementação em Python

A escolha de Python como linguagem de implementação oferece várias vantagens e algumas desvantagens. Python é amplamente reconhecido por sua facilidade de uso, sintaxe intuitiva e vasta biblioteca de pacotes que facilitam o desenvolvimento de algoritmos complexos, como o Simplex.

A biblioteca SciPy, utilizada neste projeto, fornece uma implementação otimizada do Simplex e outras ferramentas de otimização. Embora escrita em Python, SciPy faz uso intensivo de bibliotecas subjacentes escritas em C, como BLAS e LAPACK, o que garante alta eficiência computacional para operações matriciais e cálculos numéricos.

Por outro lado, a natureza interpretada de Python pode levar a tempos de execução mais lentos em comparação com linguagens compiladas, especialmente em implementações personalizadas de algoritmos que não fazem uso de bibliotecas otimizadas. Apesar disso, a integração com bibliotecas otimizadas permite mitigar parte dessa desvantagem, combinando a flexibilidade de Python com a eficiência do código nativo em C. Essa combinação torna Python uma escolha prática para projetos que exigem desenvolvimento rápido, análise estatística e experimentação com múltiplos cenários.

4.2 Tempos de Execução e Número de Iterações Médios por algoritmo



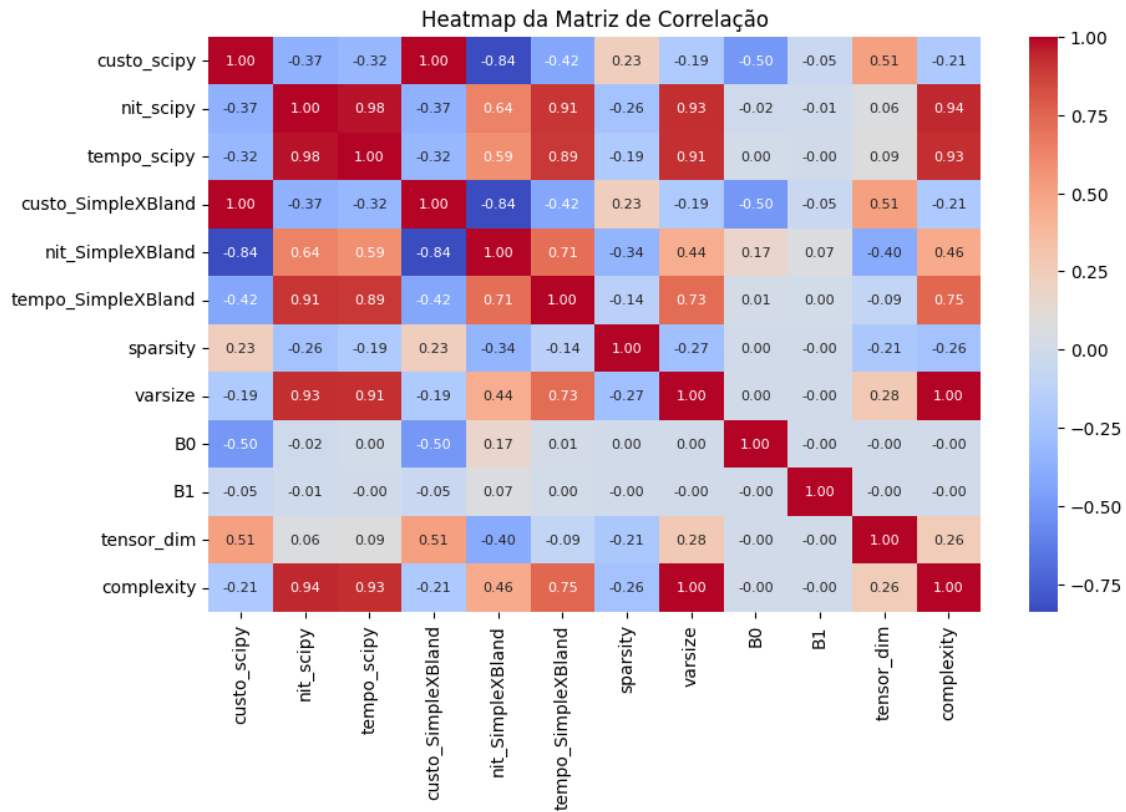
Esse primeiro gráfico apresentado é um boxplot que compara o tempo de execução e número de iterações até encontrar a solução ótima de dois algoritmos de otimização linear, o Simplex Bland,

método implementado, e o método disponível no SciPy, variando apenas o algoritmo utilizado, como um geral uma amostra projetada com parâmetros específicos para as observações do Simplex.

Um aspecto a ser notado é em relação a variabilidade e consistência dos algoritmos. O Simplex Bland apresenta maior dispersão nas iterações realizadas e tempo de execução, indicando uma sensibilidade mais elevada à natureza dos problemas analisados. Essa variabilidade pode ser atribuída à maneira como o algoritmo seleciona e explora os vértices no espaço das soluções. O número elevado de outliers nos boxplots do Simplex Bland reforça essa hipótese, indicando que para algumas instâncias, o algoritmo pode realizar um número desproporcionalmente alto de iterações.

Vale ressaltar que nas observações sobre iterações destacamos que a variabilidade do método implementado pode ser mais econômica para casos menores. É possível que as iterações da nossa implementação possuam maiores tempos, enquanto o método da biblioteca tenta otimizar o caso de início, aumentando o custo computacional para casos menores, além de modificar a quantidade de iterações realizadas.

4.3 Correlação



Nessa visualização podemos observar a correlação entre as features que descrevem o problema. Em especial, estamos interessados nos tempos de execução. Para essa análise é importante lembrar que há uma grande variabilidade intrínseca dos resultados, mesmo conservando todos os parâmetros constantes, assim como visto no tópico 4.2. Por tal motivo, é difícil atingir correlações muito altas.

Nota-se que a distribuição dos valores das restrições (regidos pela média B0 e desvio padrão B1) não afetam os tempos de execução.

A feature 'complexity' apresentada nessa visualização é dada pela complexidade esperada do pior caso considerando o tamanho da matriz inicial e sua esparsidade. Nota-se que esse parâmetro teve uma grande correlação com o tempo, confirmando os resultados previstos.

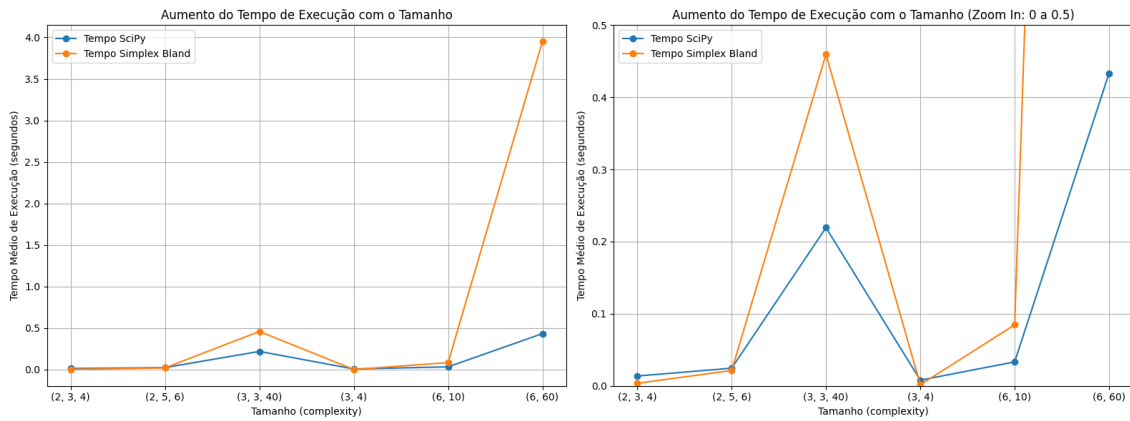
Algumas relações observadas são devidas a fatores indiretos. É possível observar uma relação

negativa forte entre a quantidade de iterações e o custo ótimo do algoritmo. A princípio essas features são independentes, porém, percebe-se que com o aumento do problema (complexidade), o custo ótimo tende a diminuir mais (mais negativo), uma vez que existem mais valores de relevância para serem maximizados, ao mesmo tempo, o aumento da complexidade causa o aumento do número de iterações. Assim, totaliza-se uma correlação negativa forte entre essas duas features, a princípio desconexas. De maneira similiar, a média das restrições B0, limita cada vez mais a relevância global atingida, visto que uma média baixa indica uma grande limitação para executar tarefas e, por isso, nota-se essa correlação negativa.

A análise por esparsidade foi bem comprometida pela alteração dos demais parâmetros em conjunto que são considerados nesse gráfico de correlação. Por isso, os resultados para a esparsidade serão analisados posteriormente em uma seção dedicada.

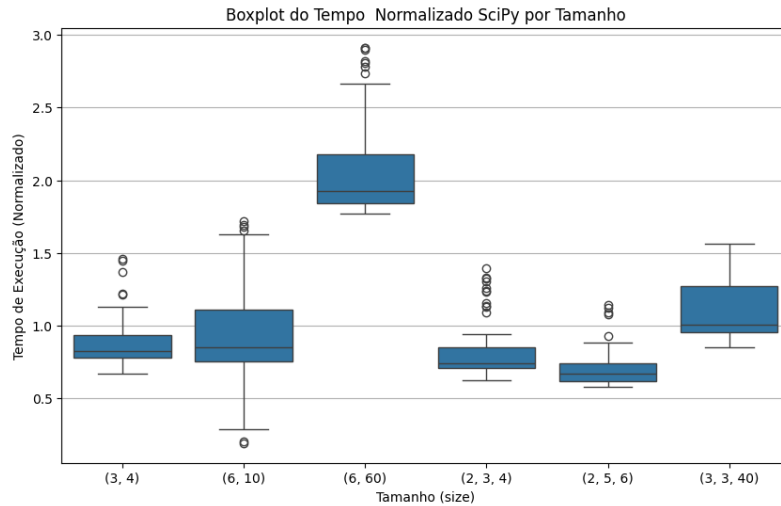
Além disso, vale notar que a correlação entre os custos é igual a 1, já que os dois alcançam a solução ótima, logo, os dois possuem o mesmo custo.

4.4 Variante: Size



O gráfico apresentado compara o desempenho dos dois algoritmos SimpleX, em termos de tempo médio de execução à medida que a complexidade do problema aumenta. O eixo X representa o tamanho e a complexidade dos problemas (em categorias como (2, 3, 4), (6, 10), etc.), enquanto o eixo Y mostra o tempo médio de execução (segundos).

Ao observar o gráfico, pode-se notar que o tempo de execução aumenta de forma mais controlada, demonstrando uma eficiência melhor em problemas maiores no scipy, enquanto o SimpleX Bland tem um aumento maior, como pode-se ver na subida súbita que ele dá em (6, 60), indicando um comportamento menos eficiente nesse problemas mais complexos. Contudo, vale notar que os dois possuem desempenhos semelhantes em problemas pequenos ou de complexidade, com o SimpleX Bland sendo até melhor em casos pequenos.

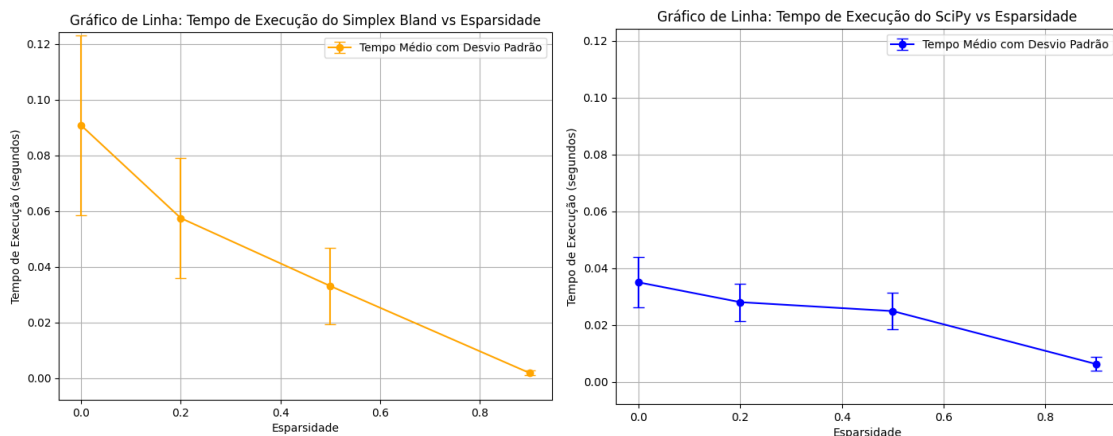


Nesse gráfico, é apresentado a tempo de execução dividido pela complexidade do pior caso para diferentes tamanho de problema inicial. Além disso, os resultados foram divididos pela média desse resultado para facilitar a comparação entre os tamanhos.

Essa visualização confirma algo que já era esperado: a relação entre a complexidade e o tempo de execução. Por isso o boxes ficam praticamente centralizados em torno de 1. O ponto importante a se analisar é o tamanho (6, 60) que teve desempenho bem distante do esperado, o que já havia sido observado na análise anterior.

Uma possível causa para o caso desse tamanho em específico é que esse problema atinge o pior caso, enquanto os demais são executados em complexidade média. Isso é plausível, uma vez que a complexidade do pior caso (dado $R_{6,60}$) é mais de 10x maior que a de melhor caso. Entretanto, o motivo desse exemplo atingir piores casos não foi descoberto e análises futuras para elucidar essa questão podem ser exploradas posteriormente em um trabalho futuro. Outra questão possível a ser analisada, comum ao medir tempos no Google Colaboratory, é a gestão de memória automática e o *Garbage Collection* que afetam os tempos de maneira imprevisível.

4.5 Variante: Sparsity



O gráfico apresentado compara o desempenho dos dois algoritmos SimpleX, ao resolver problemas de otimização em matrizes com diferentes níveis de esparsidade (densidade relativa de zeros). O Eixo X mede a proporção de zeros na matriz, variando de 0 a 1 e o Y mede o tempo médio de execução em segundos, mostrando como o tempo varia com a esparsidade.

O scipy apresenta tempos consistentemente menores em geral, mas o tempo médio diminui levemente à medida que a espacidade aumenta, enquanto o SimpleX Bland inicia com tempos mais altos para matrizes densas e tem uma queda mais acelerada, indicando que ela é mais sensível a esparsidade.

Percebe-se que há uma relação linear entre a entre a esparsidade e a complexidade esperada. Podemos manipular a expressão:

$$O((n+m)(nm(1-s) + n + m)) = O((n+m)(nm + n + m) - s(n+m)(nm))$$

Assim, $(n+m)(nm + n + m)$ representa o coeficiente constante da reta, enquanto $(n+m)(nm)$ representa o coeficiente angular, em uma função linear de s .

4.6 Teste de Escalabilidade do Problema

Foram realizados testes para verificar o limite dos simplex utilizados, aumentando as dimensões do problema e limitando o tempo em uma hora de execução. Foram feitos dois testes, um com $R_{50,50}$ no problema original, e outro com $R_{100,100}$ no problema original.

Nos dois testes, o simplex revisado disponibilizado pela biblioteca SciPy chegou ao limite de mil interações e parou a execução do algoritmo no meio, não chegando ao resultado final ótimo. Em contra partida o simplex com a regra de bland chegou no resultado ótimo no teste menor, gastando 10min10s, realizando 4449 interações, porém no segundo teste o problema alcançou a marca de uma hora sem conseguir terminar o algoritmo e, assim, finalizou o teste sem mostrar o resultado ótimo.

5 Conclusão

Os resultados obtidos com os dois algoritmos, tanto o implementado quanto o disponibilizado pela biblioteca SciPy, foram equivalentes em termos de solução ótima. No entanto, o algoritmo da biblioteca mostrou-se significativamente mais otimizado e consistente em relação ao tempo de execução, apresentando menor variabilidade mesmo para problemas mais complexos. Essa estabilidade pode ser atribuída à utilização de implementações otimizadas em outras linguagens de nível mais baixo, como C, que garantem um desempenho mais eficiente ao lidar com grandes volumes de cálculos.

Os resultados apresentados demonstram que diversas características do problema influenciam diretamente o desempenho dos algoritmos de otimização linear no problema proposto, tanto em termos de tempo de execução quanto de número de iterações necessárias para encontrar a solução ótima.

O aumento do tamanho das matrizes e da complexidade esperada do pior caso resulta em maior tempo de execução e número de iterações. Já os parâmetros relacionados à distribuição das restrições, como a média e o desvio padrão, não afetaram a escalabilidade do problema, apesar de alterarem o custo final. A esparsidade demonstrou ser um fator relevante no desempenho computacional. Matrizes mais esparsas, com maior proporção de zeros, tendem a facilitar a execução dos algoritmos devido à redução na quantidade de cálculos necessários, impactando positivamente os tempos de execução.

A análise dos fatores citados é essencial para compreender as limitações e potencialidades dos métodos de otimização linear em diferentes contextos. Trabalhos futuros podem investigar casos específicos que apresentem comportamento anômalo, como problemas que atingem o pior caso ou cuja variabilidade exacerbada ainda não está completamente compreendida.

References

- [1] Marina Andretta (2024). *Slides de Otimização Linear e aulas expositivas*. Universidade de São Paulo. Material apresentado em sala de aula e disponível na plataforma e-Disciplinas (Moodle).
- [2] SciPy (2024). *Documentação de `scipy.optimize.linprog` (Simplex Method)*. Disponível em: <https://docs.scipy.org/doc/scipy/reference/optimize.linprog-simplex.html>. Acesso em: 12 de novembro de 2024.
- [3] Dimitris Bertsimas e John N. Tsitsiklis (1997). *Introduction to Linear Optimization*. Athena Scientific, Belmont, MA, USA.
- [4] Victor Pan (1985). *On the Complexity of a Pivot Step of the Revised Simplex Algorithm*. Computers & Mathematics with Applications. Pergamon Press.